Task-2: Model.

1. Model Selection and Reasoning behind it.

As mentioned in the instructions of task-2 pretrained model can be taken from BDD100k model zoo repo (https://github.com/SysCV/bdd100k-models/tree/main/det) although repo contains good benchmarking and comprehensive details about the BDD100k object detection model but the support to download the pretrained weight is not working for now (27 Feb 2026). The URL hosting all model zoo repo is down (https://dl.cv.ethz.ch/).

Out of the listed benchmarking of model in model zoo repository I would have preferred "ConvNext" model (Backbone: ConvNeXt-S, Network: Cascade R-CNN) reason being that this model is performing best in testing and validation data when compared among the other models present in the repo. (Aim being to get the best performance on the dataset).

ConvNeXt

A ConvNet for the 2020s [CVPR 2022]

Authors: Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, Saining Xie

▸ Abstract

Results

Backbone	Network	FP16	Lr schd	MS-train	Box AP-val	Scores-val	Box AP-test	Scores-test	Config	Weights	Preds	Visuals
ConvNeXt-T	Faster R-CNN	✓	1x		33.32	scores	31.96	scores	config	model | MD5	preds	visuals
ConvNeXt-T	Faster R-CNN	✓	3x	✓	34.08	scores	33.59	scores	config	model | MD5	preds	visuals
ConvNeXt-S	Faster R-CNN	✓	3x	✓	34.79	scores	34.49	scores	config	model | MD5	preds	visuals
ConvNeXt-B	Faster R-CNN	✓	3x	✓	33.92	scores	33.30	scores	config	model | MD5	preds	visuals
ConvNeXt-T	Cascade R-CNN	✓	1x		35.51	scores	33.99	scores	config	model | MD5	preds	visuals
ConvNeXt-T	Cascade R-CNN	✓	3x	✓	35.84	scores	35.35	scores	config	model | MD5	preds	visuals
ConvNeXt-S	Cascade R-CNN	✓	3x	✓	36.11	scores	35.43	scores	config	model | MD5	preds	visuals
ConvNeXt-B	Cascade R-CNN	✓	3x	✓	35.77	scores	34.95	scores	config	model | MD5	preds	visuals

But in the real scenario, the choice will depend on the problem statement and other requirements (Hardware available, Performance required, Acceptable Latency, Inference speed etc). Depend on the available resource available best suitable model will be selected. E.g. If requirement is to infer at low-latency/real-time speed but hardware available is having limited space and compute then models with heavy backbone cannot be selected even though they excel at the object detection task.

Architecture Explanation: ConvNext + Cascade R-CNN

ConvNeXt is a pure convolutional neural network that modernizes the standard ResNet architecture. (to competes with Transformer based models)

i. **Patchify Stem:** It uses a `4x4` non-overlapping convolution with a stride of `4` to immediately down sample the image and project it into 96 channels.
ii. **Macro-Architecture:** The network processes the image through four distinct stages, progressively reducing spatial resolution while increasing channel dimensions, the blocks are distributed across the four stages as [3, 3, 27, 3]. Between each stage, separate down sampling layers (a `2x2` convolution with a stride of `2` and Layer Normalization) are applied.

iii. **Micro-Architecture (The ConvNeXt Block):** The internal design of each block introduces several critical modernizations:

- **Large Kernel Sizes:** It utilizes a `7x7` depth wise convolution to capture a larger spatial receptive field, mimicking the self-attention window of Transformers.
- **Inverted Bottleneck Design:** The block expands the channel dimension by a factor of 4 using a `1x1` convolution before reducing it back, moving the spatial feature learning to a higher dimensional space.
- **Modern Normalization and Activations:** Traditional Batch Normalization is replaced with Layer Normalization (LayerNorm), and ReLU is replaced with the smoother GELU activation function.

While standard Faster R-CNN uses a single detection head, Cascade R-CNN employs a multi-stage architecture to address the problem of noisy bounding box predictions and overfitting at high Intersection over Union (IoU) thresholds.

i. **Feature Integration (FPN & RPN):** The multi-scale feature maps from the four stages of ConvNeXt-S are first fed into a Feature Pyramid Network (FPN). The FPN fuses high-resolution, low-level features with low-resolution, high-level features. These fused maps are then passed to the Region Proposal Network (RPN), which generates initial, rough object proposals.

ii. **Cascading RoI Heads:** The core innovation of Cascade R-CNN is its sequence of Region of Interest (RoI) heads. Instead of classifying and bounding the object once, it refines the RPN proposals through typically three sequential stages:

- **Stage 1:** Operates at a standard IoU threshold (e.g., 0.5) to filter initial proposals and adjust bounding boxes.
- **Stage 2:** Uses the adjusted boxes from Stage 1 to extract new features via RoI Align. It operates at a stricter IoU threshold (e.g., 0.6) to produce tighter boxes.
- **Stage 3:** Takes the output from Stage 2 and applies the strictest IoU threshold (e.g., 0.7) for final, highly precise classification and localization.

In operation, an image is passed through the ConvNeXt-S backbone, which applies its modernized, transformer-inspired convolutions to generate robust, multi-scale feature representations. These features are aggregated by the FPN and used by the RPN to propose regions of interest. Finally, the Cascade R-CNN heads iteratively process these regions, filtering out false positives and tightening bounding box coordinates across multiple stages.

2. Fine-Tuning custom models.

As stated above selecting a model depends on the requirement and compute available we will be selecting a model. As the above BDD100K repository is not working I have decided to train my own models using the BDD100K dataset.

i. **Yolo11s:** This model is trained with complete BDD100K dataset in Kaggle environment for 100 epochs and the best performing model on validation was selected.

Architecture Explanation of Yolo11n:

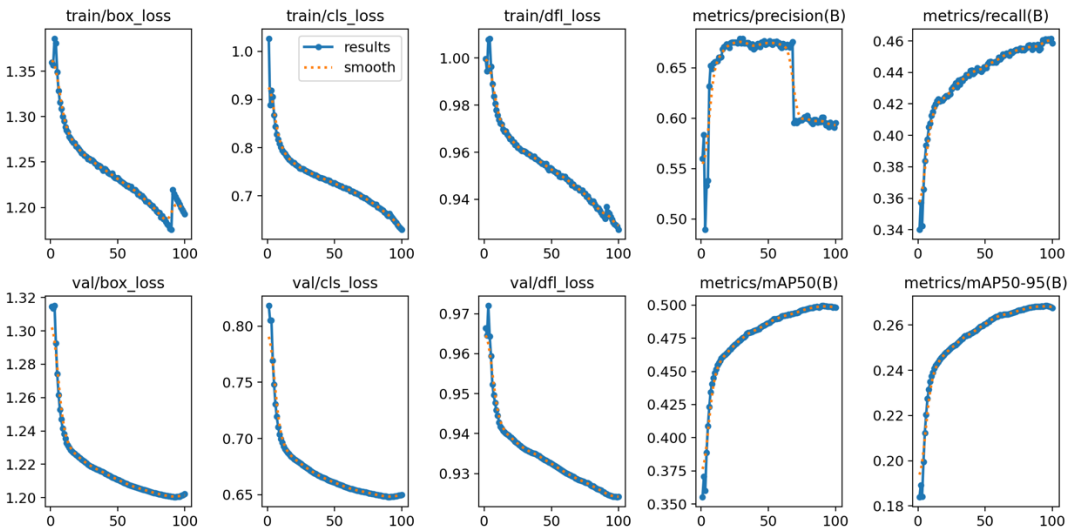
Unlike two-stage detectors like ConvNext, it is a single stage object detection model. This allows YOLO11n to run inference at low latency making it good for edge computing.

i. **Feature Extraction:** The backbone processes the input image to extract hierarchical features using two major modernizations. First, it utilizes new C3k2 blocks to improve gradient flow and capture complex visual patterns while keeping the overall parameter count low. Second, it integrates C2PSA (Cross-Stage Partial Spatial Attention), a mechanism that forces the network to focus on critical object features while actively suppressing background noise.

ii. **Feature Fusion:** The neck aggregates features from different layers of the backbone to ensure the model can detect objects of vastly different sizes. It combines a Feature Pyramid Network (FPN) to pass high-level semantic information downwards with a Path Aggregation Network (PAN) to pass low-level spatial information upwards. These fused maps are then processed by the C3k2 blocks to maintain a robust, multi-scale representation before making predictions.

iii. Prediction: The head outputs the final object categories and locations. It uses an anchor-free design without fixed templates, making it flexible for objects of any size. It also features a "decoupled" structure that splits the work into two separate paths: one identifies what the object is, and the other calculates exactly where it is.

Training plots:



ii. **RF-DETR:** This model is trained using a subset of BDD100k (10% of it). Subset strategy ensure to represent whole BDD100k distribution pattern in the subset data. 12 epoch of training has been done in Nvidia Jetson Thor Edge device (you may

face errors when deploying same script/requirement file as Jetson Thor is an arm64 device where we mostly use x86 based systems).

Architecture Explanation of RF-DETR:

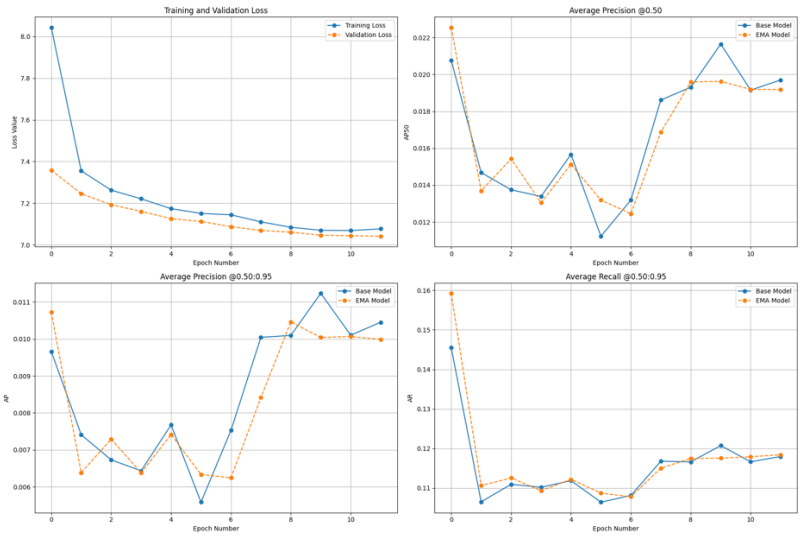
Pure Transformer based object detection model developed by Roboflow.

Feature Extraction: Uses pretrained DINOv2 vision transformer to gain self-supervised understanding. It uses single-scale feature map rather than a heavy multiscale feature pyramid which introduces latency. Use of foundational model makes the model incredibly adaptable to custom tasks with limited datasets.

Projector/Feature Bridging: A projector module connects the vision transformer encoder to the detection decoder. It formats single scale features for faster speed and utilises Layer normalisation instead of batch normalisation to stable training.

Head: Generates final predictions using transformer decoder. It uses deformable attention which samples only a small set of critical key points around an object to save compute. No need of non-maximum suppression on post processing.

Training plots:



Quick comparison of benchmarks of these models on COCO dataset for object detection.

Benchmark shows RF-DETR-M outperforms both ConvNext and YOLO11n on the performance.

Performance: RF-DETR > YOLO11n > ConvNext.
Latency: YOLO11n> RF-DETR.

name	Pretrained Model	Method	Lr Schd	box mAP	mask mAP	#params	FLOPs	Fine-tuned Model
ConvNeXt-T	ImageNet-1K	Mask R-CNN	3x	46.2	41.7	48M	262G	model
ConvNeXt-T	ImageNet-1K	Cascade Mask R-CNN	3x	50.4	43.7	86M	741G	model
ConvNeXt-S	ImageNet-1K	Cascade Mask R-CNN	3x	51.9	45.0	108M	827G	model
ConvNeXt-B	ImageNet-1K	Cascade Mask R-CNN	3x	52.7	45.6	146M	964G	model
ConvNeXt-B	ImageNet-22K	Cascade Mask R-CNN	3x	54.0	46.9	146M	964G	model
ConvNeXt-L	ImageNet-22K	Cascade Mask R-CNN	3x	54.8	47.6	255M	1354G	model
ConvNeXt-XL	ImageNet-22K	Cascade Mask R-CNN	3x	55.2	47.7	407M	1898G	model

Source: https://github.com/facebookresearch/ConvNeXt/tree/main/object_detection

Architecture	COCO AP_{50}	COCO $AP_{50:95}$	RF100VL AP_{50}	RF100VL $AP_{50:95}$	Latency (ms)	Params (M)
RF-DETR-N	67.6	48.4	84.1	57.1	2.32	30.5
RF-DETR-S	72.1	53.0	85.9	59.6	3.52	32.1
RF-DETR-M	73.6	54.7	86.6	60.6	4.52	33.7
YOLO11-N	52.0	37.4	81.4	55.3	2.49	2.6
YOLO11-S	59.7	44.4	82.3	56.2	3.16	9.4
YOLO11-M	64.1	48.6	82.5	56.5	5.13	20.1
YOLO11-L	65.3	50.2	x	x	6.65	25.3
YOLO11-X	66.5	51.2	x	x	11.92	56.9

Source: https://rfdetr.roboflow.com/learn/benchmarks